Unified Backend

One clean, multi-tenant backend to replace iboplayer, bobplayer and cr-unified

Team proposal

Where we are

3

backends copied from the same original, drifting apart

~100k

lines of code, controllers up to 9,000 lines

3×

every fix has to be made up to three times

The rule we can't break

Installed apps can't be updated.
They must keep calling the same URLs, with the same payloads, and get the same responses.

Samsung · LG · Android · iOS · Roku · set-top boxes — and the external store

What's wrong today

...and how the new backend fixes it

	TODAY	NEW BACKEND
Code	Giant controllers, same logic copied many times	Small modules, each rule written once, tested
Apps	Hardcoded `if (tenant === …)` branches	Apps are configuration, not code
Data	Dates as text, money as floats, mixed types, duplicates	Real types, unique MAC, clean migration
Security	Hardcoded secrets, unverified payments, open routes	Secret store, verified payments, auth on every route
Reliability	Jobs run up to 24×, requests hang, processes crash	Jobs run once, every request gets an answer

The proposal

CODE

One backend

Serves every app from a single codebase

DATA

Central DB + one DB per app

Shared platform data in one place, each app isolated

INTERFACES

API only

Admin, reseller and customer UIs become separate frontends

Two phases

PHASE 1

Faithful clean rebuild

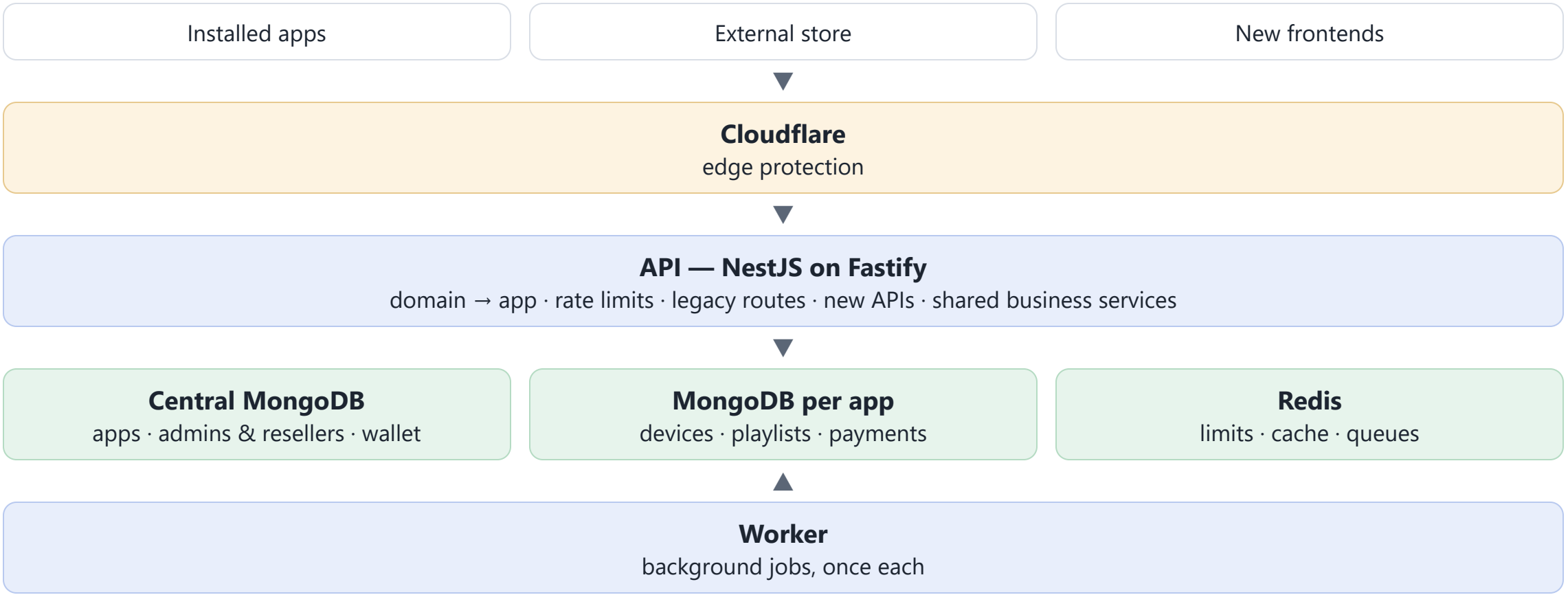
New architecture, clean code — every contract apps and the store see stays identical. Bugs we must copy are tagged in the code.

PHASE 2 · BEFORE GO-LIVE

Fix & clean up

Fix every tagged bug and security gap, remove dead endpoints after checking them against the real apps.

Architecture



Stopping floods & fake devices

Edge
Cloudflare rules, trusted client IPs

→

Rate limits
shared across all servers

→

Pending device
new MAC waits in an auto-expiring list

→

Real action?
playlist, payment → becomes a device

Fake devices never add a playlist → they **expire on their own**. No mass deletions, no permanent bans.

The stack

Node.js 24
same language, current LTS

TypeScript
catches undefined & typo bugs before runtime

NestJS
structure: modules, guards, testing

Fastify
2–3× faster than Express

MongoDB
data is already there — easy migration

Redis
shared limits, cache, locks

BullMQ
jobs run once, retry, survive restarts

zod
validates config & all input

pino
structured logs

Docker
same environment everywhere

Tests + CI
nothing merges unless green

Cloudflare
edge protection

Roadmap

Discover
endpoints, contracts, data

Foundation
project, infra, tests

Core
tenancy, devices, licenses, wallet

Compatibility
legacy routes, store, payments

Migrate & harden
data migration, load & security tests

Phase 2
fixes & cleanup

Cutover
app by app, with rollback

Questions?

One backend · same contracts · clean code